Closing the Gap: Trellis-Coded Quantization for KV Cache at 2–3 Bits

buun[†], Claude (Anthropic)

Abstract

We present the first application of trellis-coded quantization (TCQ) to key-value cache compression in large language model inference. By combining Fast Walsh-Hadamard rotation with Viterbi-optimal trellis encoding, our method achieves 10–44% KL-divergence reduction over scalar quantization at 2–3 bits per value, with $O(1)$ parallel decode. At 3.25 bits per value, our method produces *lower* perplexity than uncompressed FP16 KV cache due to a beneficial norm-scaling effect. At 2.25 bits per value, TCQ significantly improves 2-bit quantization, closing the gap with 3-bit scalar methods. We further identify a context-dependent codebook quality phenomenon: the relationship between codebook MSE and downstream model quality inverts between short and long contexts, explained by CLT averaging of quantization error in the attention mechanism.

[†]**Correspondence:** `https://x.com/spiritbuun`
Code: `https://github.com/spiritbuun/llama-cpp-turboquant-cuda`

1 Introduction

Long-context inference is central to modern large language model (LLM) applications — from document analysis and code generation to multi-turn dialogue. The key-value (KV) cache, which stores intermediate attention states to avoid redundant computation, grows linearly with sequence length and dominates GPU memory consumption during inference. For a 27-billion-parameter model at 128K context, the KV cache alone exceeds 40 GB in FP16, often surpassing the model weights themselves.

Quantizing the KV cache to low bit widths (2–4 bits per value) is the most direct approach to this memory bottleneck. Recent work has made significant progress using scalar quantization [15, 16] and online vector quantization with random rotation [1]. However, all existing methods quantize each element or vector independently: they do not exploit sequential structure within the KV vectors to achieve compression beyond what independent coding permits.

In the signal processing literature, *trellis-coded quantization* (TCQ) [2] has long been known to close most of the 1.53 dB gap between scalar and optimal vector quantization for Gaussian sources. TCQ constrains the sequence of quantization indices to follow a finite-state trellis, enabling a much larger effective codebook at the same bit rate. The Viterbi algorithm finds the globally optimal encoding, and — crucially for GPU inference — the bitshift trellis structure allows each element to be decoded independently in $O(1)$ via a sliding bit window.

Despite TCQ's theoretical appeal, it has not been applied to KV cache compression. The only prior application of TCQ to neural networks is QTIP's use for weight quantization [10], where the encoding cost is amortized at model load time. For KV cache, encoding must happen online during inference, demanding a lean trellis and efficient CUDA implementation.

In this work, we combine Fast Walsh-Hadamard rotation — which transforms KV vectors into an i.i.d. Gaussian distribution amenable to analytical codebook design — with Viterbi-optimal trellis encoding using a 512-state bitshift trellis. Our method stores only 3.25 bits per value (3-bit) or 2.25 bits per value (2-bit) while achieving substantial quality improvements over scalar quantization at the same bit rate.

Contributions. We make four contributions:

1. **TCQ for KV cache.** We present the first application of trellis-coded quantization to key-value cache compression, achieving 10–44% KL-divergence reduction over scalar quantization at 2–3 bits per value (Section 3). At 3.25 bpv, our method produces lower perplexity than uncompressed FP16 due to a beneficial norm-scaling effect. At 2.25 bpv, TCQ substantially improves 2-bit quantization, reducing the PPL gap from +9% to +1.2% relative to FP16.
2. **Context-dependent codebook quality.** We identify and explain a phenomenon where the relationship between codebook MSE and model quality inverts between short and long contexts (Section 4.3). At short contexts, codebook *regularity* determines quality; at long contexts, *MSE* dominates via CLT averaging of quantization error in the attention mechanism. No single codebook is optimal across all context lengths.
3. **MSE–quality divergence.** We demonstrate that every aggregate error metric we tested (13 statistics including covariance structure, kurtosis, tail behavior, and simulated attention error) improves monotonically with codebook training iterations, yet model perplexity degrades beyond 10 iterations (Section 4.2). This provides empirical evidence for the theoretical prediction that MSE is the wrong objective for quantization in attention [6].
4. **Query anisotropy.** We measure extreme anisotropy in the rotated query space (effective rank ~5 out of 128 dimensions), implying that 96% of quantization error is invisible to the attention mechanism (Section 4.4). Current TCQ errors are isotropic in the query eigenbasis, suggesting a 25× theoretical ceiling for query-aware codebook design.

Our implementation is open-source, integrated into llama.cpp with full CUDA support for both encoding and flash-attention decoding.

2 Background and Related Work

We situate our work at the intersection of three lines of research: KV cache compression for efficient LLM inference, trellis-coded quantization from the signal processing literature, and random rotation techniques for quantization.

2.1 KV Cache Compression

The key-value cache stores intermediate attention states across all layers and grows linearly with sequence length, consuming the majority of GPU memory during long-context inference. For a model with L layers, h attention heads, head dimension d, and sequence length n, the KV cache requires $2Lhdn$ elements in FP16 — over 40 GB for a 27B-parameter model at 128K context.

Scalar quantization. The simplest approach quantizes each KV element independently. llama.cpp's built-in types (q4_0, q8_0) use per-block quantization with block sizes of 32, achieving 4–8 bits per value with modest quality loss [14]. KIVI [15] applies per-channel quantization with 2-bit keys and 2-bit values, using residual compensation to maintain quality. KVQuant [16] adds per-channel sensitivity weighting and non-uniform quantization for sub-4-bit operation.

Vector quantization. TurboQuant [1] introduced online vector quantization for KV cache, using random rotation to standardize the distribution before applying analytically-computed Lloyd-Max codebooks. This eliminates the need for calibration data and achieves near-optimal distortion at 3–4 bits per value. VecInfer [17] combines smooth transforms with Hadamard rotation and VQ, with fused CUDA kernels achieving 2.7× speedup. NSNQuant [18] uses triple normalization to align distributions to standard normal, enabling calibration-free codebooks — an observation that aligns with our post-FWHT Gaussian finding.

Transform coding. KVTC [19] applies classical transform coding (PCA decorrelation + adaptive quantization + entropy coding) to achieve up to 20× compression. Their PCA-based decorrelation is data-dependent and requires calibration, in contrast to the data-independent FWHT rotation used in our work and TurboQuant.

Bit allocation. "More Keys, Less Values" [20] proves theoretically that keys have higher spectral norms than values, motivating asymmetric bit budgets (e.g., 4-bit keys with 2-bit values) — a finding we validate empirically in Section 5.

Our position. All of the above use either scalar quantization or independent vector quantization — each element or vector is quantized without considering its neighbors in the sequence. We introduce *trellis-coded* quantization, which jointly optimizes the quantization of 128 consecutive elements via the Viterbi algorithm, achieving a larger effective codebook at the same bit rate. To our knowledge, no prior work has applied TCQ to KV cache compression.

2.2 Trellis-Coded Quantization

Trellis-coded quantization was introduced by Marcellin and Fischer [2] as the source-coding dual of Ungerboeck's trellis-coded modulation (TCM). The key idea: by constraining the sequence of quantization indices to follow a trellis (finite-state machine), one can use a larger codebook than the bit rate would otherwise permit, with the trellis structure ensuring decodability.

Coding gain. For an i.i.d. Gaussian source, a scalar quantizer at rate R bits/sample incurs a distortion penalty of 1.53 dB relative to the Shannon rate-distortion bound $D(R) = \sigma^2 \cdot 2^{-2R}$. This gap is fundamental to one-dimensional quantization [13]. TCQ recovers most of this gap: an 8-state TCQ achieves ∼0.5 dB from $R(D)$, and a 256-state TCQ achieves ∼0.2 dB [3]. The gain comes from the trellis effectively creating a high-dimensional quantizer from scalar operations — the "effective dimension" grows with trellis memory [12, 24].

Structure. A TCQ with 2^L states and rate k bits/element has a codebook of 2^L reconstruction values, organized into 2^k cosets via set partitioning. At each time step, the trellis state determines which cosets are reachable; within each reachable coset, the nearest codeword is selected. The Viterbi algorithm finds the globally optimal path in $O(2^L \cdot n)$ time.

Universal TCQ. Kasner and Marcellin [4] showed that TCQ with *uniform thresholds* (no training) achieves performance "comparable with fully optimized entropy-constrained TCQ" at most encoding rates. This remarkable result implies that the coding gain comes primarily from the trellis structure and set partitioning, not from codeword optimization. However, as we show in Section 4.3, this universality breaks down at very low rates (2-bit) where codebook training is essential.

TCQ for neural network quantization. QTIP [10] recently applied TCQ to LLM weight quantization with impressive results at 2–4 bits per weight. QTIP uses a bitshift trellis with 2^{16} states, hash-based codebook mapping, and Hessian-weighted distortion (LDLQ). Our work differs in three respects: (1) we quantize activations (KV cache) rather than weights, requiring online encoding at inference time; (2) we use a much smaller trellis ($2^9 = 512$ states) to keep encode latency manageable; (3) we exploit the bitshift trellis's sliding-window property for $O(1)$ parallel GPU decode, which is not needed for weight dequantization (done once at load time).

2.3 Random Rotation for Quantization

Channel-wise outliers in LLM activations cause severe quantization degradation: a few high-magnitude channels dominate the dynamic range, starving other channels of precision. Random orthogonal rotation redistributes energy uniformly across channels, eliminating outliers at the cost of one matrix-vector multiply.

Theory. For a unit-norm vector $\mathbf{x} \in \mathbb{R}^d$, the coordinates of $R\mathbf{x}$ (where R is drawn uniformly from the orthogonal group) are approximately i.i.d. $\mathcal{N}(0, 1/d)$ for large d, by concentration of measure on the sphere. The Fast Walsh-Hadamard Transform with random sign flips provides an efficient $O(d \log d)$ approximation to a random rotation [23].

Applications. QuIP# [11] combines random Hadamard rotation with E8 lattice codebooks for weight quantization, demonstrating that rotation + structured codebook outperforms trained codebooks despite the latter achieving lower MSE on training data. This finding, grounded in the Zamir-Feder theorem that

lattice quantization noise is approximately white [5], parallels our observation that codebook regularity matters more than MSE at short contexts (Section 4.3).

TurboQuant [1] applies FWHT rotation to KV cache quantization and proves that the resulting distribution admits analytically optimal scalar quantizers within 2.72× of the Shannon bound — no training data required. RotateKV [21] extends this with channel reordering before FWHT to further reduce outlier impact. MixQuant [22] provides the first systematic analysis of outlier suppression under block Hadamard rotations, showing that outliers are minimized when pre-rotation L_1 mass is evenly distributed.

Our contribution. We combine FWHT rotation (which makes the data amenable to analytical codebook design) with TCQ (which provides coding gain beyond scalar quantization). The rotation converts the KV quantization problem from an intractable high-dimensional distribution to a well-understood i.i.d. Gaussian, for which TCQ's theoretical properties are fully characterized [2]. This combination — rotation for distribution standardization, trellis coding for compression efficiency — has not been previously explored for KV cache or, to our knowledge, for any online activation quantization setting.

3 Method

We describe our approach to compressing the KV cache using trellis-coded quantization. The pipeline processes each 128-element group of a key or value vector through five stages: (1) Walsh-Hadamard rotation, (2) norm extraction, (3) Viterbi trellis encoding, (4) bitpacked storage, and (5) sliding-window decode during attention. Figure **??** illustrates the full pipeline.

3.1 Walsh-Hadamard Rotation

Raw KV vectors exhibit highly non-uniform channel distributions: a few channels carry most of the energy while others are near-zero. Directly quantizing such vectors wastes bits on low-energy channels and underserves high-energy ones.

Following TurboQuant [1], we apply the Fast Walsh-Hadamard Transform (FWHT) with random sign flips to each 128-element group before quantization. For a group $\mathbf{x} \in \mathbb{R}^{128}$, we compute:

$$\tilde{\mathbf{x}} = \frac{1}{\sqrt{128}} H_{128}\, S\, \mathbf{x} \tag{1}$$

where H_{128} is the 128×128 Hadamard matrix (applied in $O(n \log n)$ via the butterfly algorithm) and $S = \mathrm{diag}(s_1, \ldots, s_{128})$ with $s_i \in \{-1, +1\}$ drawn from a fixed pseudorandom sequence.

After rotation, each coordinate of $\tilde{\mathbf{x}}$ is approximately i.i.d. Gaussian with variance $\|\mathbf{x}\|^2/128$. This is a consequence of the concentration of measure on the sphere: for high-dimensional vectors, random orthogonal projections yield near-Gaussian marginals.

This transformation has three critical effects:

1. **Uniformization.** All channels have equal variance, eliminating the need for per-channel scaling or mixed-precision allocation.

2. **Known distribution.** The post-rotation distribution is analytically characterized (Gaussian), enabling optimal codebook design without calibration data.

3. **Decorrelation.** Inter-channel correlations are destroyed, making the i.i.d. assumption required by TCQ theory valid in practice.

The rotation is applied once at KV cache write time and inverted once at attention compute time, adding negligible overhead to both paths.

3.2 Norm Extraction and Scaling

Before trellis encoding, we extract the L_2 norm of each 128-element group:

$$n = \|\tilde{\mathbf{x}}\|, \qquad \hat{\mathbf{x}} = \tilde{\mathbf{x}}/n \tag{2}$$

The norm n is stored as a single FP16 value shared across 128 elements (0.125 bits/element overhead). The normalized vector $\hat{\mathbf{x}}$ has unit norm with each coordinate distributed as $\mathcal{N}(0, 1/128)$.

During reconstruction, the decoded vector is rescaled by $\alpha \cdot n$, where α is a learned scaling factor. We find that the optimal α differs for keys and values:

$$\hat{\mathbf{x}}_{\text{recon}} = \begin{cases} \alpha_K \cdot n \cdot \text{decode(bits)} & \text{for keys} \\ \alpha_V \cdot n \cdot \text{decode(bits)} & \text{for values} \end{cases} \tag{3}$$

with $\alpha_K = 1.0$ and a context-adaptive α_V that adjusts based on the current sequence length n_{seq}:

$$\alpha_V^{\text{3-bit}}(n) = 1.04 - 0.01 \cdot \log_2(n/2048) \tag{4}$$

$$\alpha_V^{\text{2-bit}}(n) = 1.04 + 0.06 \cdot (1 - 2048/n) \tag{5}$$

These formulas are derived from the empirical alpha sweep in Section 5.5. The 3-bit alpha decreases with context (approaching 1.0), while the 2-bit alpha increases (asymptoting to 1.10). Alpha is applied at decode time so that all tokens in the KV cache are scaled uniformly as the context grows.

The asymmetry arises because key and value errors propagate differently through attention. Key errors enter inside the softmax exponential and are nonlinearly amplified [7], while value errors propagate linearly through the weighted average. The slight upward scaling of value norms provides a beneficial regularization effect: it sharpens the attention distribution, reducing the influence of noisy low-weight tokens. At 3.25 bits per value, this scaling is sufficient for the quantized model to achieve *lower perplexity than uncompressed FP16* (Section 5).

3.3 Trellis-Coded Quantization

3.3.1 Trellis Structure

We use a bitshift trellis with $2^L = 512$ states, rate $k = 3$ bits per element, and scalar quantization ($V = 1$). The trellis is a directed graph where state i has an edge to state j if and only if:

$$j = (i \gg k) \mid (c \ll (L - k)) \quad \text{for some } c \in \{0, 1, \ldots, 2^k - 1\} \tag{6}$$

That is, the next state is formed by right-shifting the current state by k bits and inserting k new bits at the top. Each state has $2^k = 8$ outgoing edges and $2^k = 8$ incoming edges.

Each state $s \in \{0, \ldots, 2^L - 1\}$ is associated with a reconstruction value $C[s] \in \mathbb{R}$ from a trained codebook. The codebook is organized into $2^k = 8$ *cosets*, where coset c contains all states with the same upper k bits. Set partitioning ensures that states within the same coset are well-separated in reconstruction value, analogous to Ungerboeck's trellis-coded modulation [2].

3.3.2 Viterbi Encoding

Given a normalized vector $\hat{\mathbf{x}} = (x_1, \ldots, x_{128})$, we find the trellis walk that minimizes total squared error:

$$\min_{s_1, \ldots, s_{128}} \sum_{t=1}^{128} (x_t - C[s_t])^2 \quad \text{s.t. } (s_t, s_{t+1}) \text{ is a valid edge } \forall t \tag{7}$$

This is solved exactly by the Viterbi algorithm. At each time step t, we maintain a cost vector over all 2^L states and update via:

$$V_t(j) = \min_{i:(i,j)\in E} \left[V_{t-1}(i) + (x_t - C[j])^2 \right] \tag{8}$$

After processing all 128 elements, we backtrack from the minimum-cost final state to recover the optimal state sequence.

Free initialization. Unlike classic TCQ which constrains the initial state, we allow all 2^L states as valid starting points with equal cost. This yields 37.6% additional MSE reduction at 3-bit by removing an artificial constraint on the first few elements.

Complexity. Encoding is $O(2^L \times n)$ per block, where $n = 128$. With $L = 9$, this is $128 \times 512 = 65{,}536$ state evaluations per block. The Viterbi iterations are *inherently sequential* along the time axis — this is the fundamental cost of trellis coding. In our CUDA implementation, encode uses 512 threads with barrier-synchronized iterations and runs entirely in shared memory.

3.3.3 Bitpacked Storage

The trellis walk is stored as a packed bitstream. The initial state requires L bits; each subsequent step contributes only k new bits (the edge label c from Equation 4). For 128 elements at $k = 3$ bits:

$$\text{Total} = L + 127 \times k = 9 + 381 = 390 \text{ bits} \tag{9}$$

Combined with the FP16 norm (16 bits shared across 128 elements), this gives a storage cost of $(390 + 16)/128 = 3.17$ bits per value for 3-bit TCQ. In practice, byte-alignment padding brings this to 3.25 bpv. At 2-bit ($k = 2$): $(9 + 127 \times 2 + 16)/128 = 2.23$ bpv, padded to 2.25 bpv.

3.3.4 Sliding-Window Decode

The critical property enabling GPU-efficient inference: **each element can be decoded independently in** $O(1)$.

To reconstruct element t, we read L consecutive bits starting at position $t \times k$ in the bitstream. These L bits *are* the state index s_t, because the bitshift transition rule ensures that consecutive states share $L - k$ bits:

$$s_t = \text{bits}[t \cdot k \ : \ t \cdot k + L] \tag{10}$$

The reconstructed value is then $C[s_t] \times \alpha \times n$ — a single codebook lookup and two multiplications. The 512-entry codebook (2 KB) fits comfortably in GPU shared memory, enabling coalesced access with no global memory traffic during attention computation.

This $O(1)$ decode property is fundamental: while encoding requires sequential Viterbi (done once at KV cache write time), *decoding is fully parallel* and sits on the latency-critical attention path. Each token's KV cache is decoded independently across all heads and layers.

3.4 Effective Codebook Size

A key advantage of TCQ over scalar quantization is the *effective codebook expansion.* A scalar k-bit quantizer has 2^k reconstruction levels. TCQ with 2^L states has 2^L reconstruction levels while storing only k bits per element.

At 3-bit with $L = 9$: the scalar quantizer chooses among 8 levels; TCQ chooses among 512 levels, constrained by the trellis. This 64× expansion of the effective codebook is what provides TCQ's coding gain — the ability to represent the source with finer granularity than the bit rate would otherwise permit.

The coding gain of an N-state TCQ for a Gaussian source is well-characterized in the literature [2, 3]: 8-state TCQ achieves approximately 0.5 dB from the rate-distortion bound $R(D)$; 256-state TCQ achieves approximately 0.2 dB. Our 512-state trellis operates in the regime of diminishing returns, capturing the majority of available coding gain.

Table 1: GLA iteration sweep for 3-bit TCQ codebooks. MSE improves monotonically; perplexity does not. Results on Qwen3.5-27B, wikitext-2, 64 chunks at 2K context.

GLA Iters	MSE Reduction	PPL (2K)	Δ PPL
0 (coset init)	0.1%	5.919	+1.64%
3	24.3%	5.845	+0.37%
5	33.8%	5.858	+0.58%
10	40.4%	5.939	+1.97%
20	46.0%	5.971	+2.53%
30	48.2%	5.873	+0.85%
50	52.8%	5.831	+0.13%
100	54.1%	5.889	+1.12%

4 Codebook Design and Theory

The design of TCQ codebooks for KV cache quantization reveals several phenomena not predicted by classical quantization theory. This section presents our findings on codebook training, the divergence between MSE and model quality, and the context-length dependence of optimal codebook selection.

4.1 Codebook Training via Generalized Lloyd Algorithm

Our codebook training follows the Generalized Lloyd Algorithm (GLA) applied to the TCQ trellis [8]. The procedure alternates two steps:

1. **Encode step.** Given the current codebook C, run Viterbi encoding on training data to find the optimal state sequence for each training vector.
2. **Centroid step.** For each state s, update $C[s]$ to the mean of all training samples assigned to state s.

Training data consists of i.i.d. samples from $\mathcal{N}(0, 1/128)$, matching the post-FWHT distribution. This eliminates the need for model-specific calibration data — the codebook depends only on the known post-rotation distribution.

Initialization. Codebooks are initialized with analytically-computed Lloyd-Max levels for the Gaussian distribution, arranged in monotonically increasing order within each coset. This *coset initialization* preserves the set-partitioning structure that provides TCQ's coding gain.

4.2 MSE–Quality Divergence

A central finding of our work is that **minimizing quantization MSE does not minimize downstream model quality loss**. Table 1 shows the relationship between GLA iteration count, MSE reduction, and model perplexity.

Several patterns emerge:

Early stopping is optimal. Three GLA iterations suffice to capture the trellis coding gain without degrading model quality. This result is robust across seeds and datasets: we verified with 4 independent random seeds and 3 dataset splits (test, validation, train) with 64 chunks each, confirming that 3-iteration codebooks consistently match the best known codebook ($\Delta < 0.005$ PPL).

Over-training degrades quality. Beyond 10 iterations, perplexity degrades by 0.06–0.12 despite continued MSE improvement. This degradation persists across all evaluation datasets, ruling out measurement noise.

Aggregate error metrics do not predict the degradation. We tested 13 error statistics including covariance structure, kurtosis, tail percentiles, signal-error correlation, and simulated attention divergence

Table 2: All tested aggregate error metrics improve monotonically with GLA iterations, yet none predict perplexity. Pearson correlation with PPL shown.

Metric	Direction w/ training	Pearson w/ PPL
Element-wise MSE	Improves	+0.10
Off-diagonal Σ_e ratio	Improves	+0.07
Error kurtosis	Improves	+0.05
Tail percentiles (p99.9)	Improves	+0.03
$\mathrm{tr}(\Sigma_q \cdot \Sigma_e)$	Improves	+0.05
Signal-error correlation	Improves	+0.05
Simulated attention KL	Improves	+0.05
Temperature perturbation ε	Improves	+0.05

(Table 2). *Every metric improves monotonically* with GLA iterations, yet perplexity exhibits non-monotonic behavior. The mechanism by which over-trained codebooks degrade model quality remains invisible to all single-layer aggregate analysis we attempted.

Connection to the quantization literature. This finding has theoretical grounding in three independent results: (1) Kasner & Marcellin's Universal TCQ demonstrated that untrained codebooks with uniform thresholds achieve "performance comparable with fully optimized ECTCQ" at most rates [4] — the trellis structure provides coding gain, not codeword optimization; (2) Zamir & Feder showed that lattice (structured) quantizers produce white, signal-independent noise, while trained quantizers produce signal-dependent noise that is catastrophically amplified by nonlinear operations like softmax [5]; (3) Ordentlich & Polyanskiy proved that for matrix multiplication (which attention is), MSE-optimal quantizers are 3.4× worse than task-optimal ones at realistic dimensions [6].

We hypothesize that over-training progressively destroys the coset regularity that provides TCQ's coding gain. The coset structure — monotonically interleaved reconstruction levels across trellis states — acts as an implicit regularizer that maintains error whiteness. GLA iterations, optimizing only for MSE, have no incentive to preserve this structure. We observe that monotonic coset fraction degrades from 64/64 (coset init) to 48/64 after aggressive training.

4.3 Context-Dependent Codebook Quality

Our systematic evaluation across context lengths from 2K to 128K tokens reveals that **no single codebook is optimal at all context lengths**. Rankings among codebooks — including between trained and analytical (compiled-in) codebooks, and between different training methods — shift unpredictably with context length. Codebook selection affects KLD by 2–7%, a meaningful but secondary effect compared to the 20–44% gain from TCQ itself.

We propose a two-regime model that explains *why* rankings are context-dependent, grounded in the interaction between quantization error statistics and the attention mechanism:

4.3.1 Short Context (< 8K): Variance Dominates

At short context lengths, softmax attention is concentrated on a small number of tokens (low-entropy attention distribution). Individual quantization error spikes can corrupt the attention pattern for an entire query. In this regime, *error variance* — the regularity and predictability of quantization errors — dominates model quality.

Structured codebooks (fewer GLA iterations, preserved coset regularity) produce more predictable errors. By the Zamir-Feder lattice noise theorem [5], regular quantizer structures produce approximately white, signal-independent noise. This noise is benign for softmax attention because it does not systematically shift attention weights.

4.3.2 Long Context (> 8K): Bias Dominates via CLT Averaging

At long context lengths, softmax attention distributes weight across many tokens. The output at each position is a weighted average of value vectors:

$$\mathbf{o} = \sum_{i=1}^{n} \alpha_i \mathbf{v}_i \tag{11}$$

where α_i are the softmax attention weights. The quantization error in the output is correspondingly a weighted average of per-token errors:

$$\mathbf{e}_o = \sum_{i=1}^{n} \alpha_i \mathbf{e}_i \tag{12}$$

By the central limit theorem, as the effective number of attended tokens $n_{\text{eff}} = 1/\sum_i \alpha_i^2$ grows, the *variance* of $\mathbf{e}_o$ is suppressed by a factor of $\sim 1/n_{\text{eff}}$, while the *bias* (systematic error from suboptimal codebook placement) is not:

$$\text{Var}[\mathbf{e}_o] \approx \frac{\sigma_e^2}{n_{\text{eff}}}, \qquad \mathbb{E}[\mathbf{e}_o] = \sum_i \alpha_i \mathbb{E}[\mathbf{e}_i] \neq 0 \tag{13}$$

At long context, CLT averaging suppresses the variance component of output error, changing which codebook properties matter. The relative importance of MSE (favoring trained codebooks) versus error regularity (favoring analytical codebooks) shifts with context length, producing the observed ranking instability. At 3-bit, the analytical codebook's regularity advantage is sufficient to win at moderate context lengths (8K–32K) despite higher MSE, while trained codebooks may recover at very long contexts where even the attenuated bias term matters.

4.3.3 Supporting Theory: Finite Blocklength Dispersion

The crossover is further explained by the finite-blocklength dispersion framework of Kostina & Verdú [9]. The gap between a quantizer's distortion and the rate-distortion bound $R(D)$ decomposes as:

$$D - D^*(R) \approx \underbrace{D_{\text{mismatch}}}_{\text{constant}} + \underbrace{\sqrt{\frac{V(R)}{n}} \cdot Q^{-1}(\epsilon)}_{\text{shrinks with } n} \tag{14}$$

A well-trained codebook has small D_{mismatch} and its gap is dispersion-dominated (shrinks with block length). A mismatched codebook has large, persistent D_{mismatch}. This framework predicts that codebook differences become more consequential at long contexts, where the dispersion term is small and codebook mismatch dominates — consistent with our observation that codebook rankings change with context length.

4.3.4 The 2-Bit Regime: Overload Distortion

At 2 bits per value, the codebook has only 4 reconstruction levels per coset. In this regime, *overload distortion* (clipping of tail samples) dominates granular distortion. The UTCQ universality result — that untrained codebooks suffice — breaks down because there are too few levels for the coset structure alone to adequately cover the Gaussian tails.

This explains why codebook training provides substantially larger improvements at 2-bit (up to 16% KLD reduction) than at 3-bit, where the analytical codebook is competitive at short contexts. It also explains the compiled-in 2-bit codebook's anomalous behavior at 64K tokens: the specific GLA local minimum produces error tails (higher kurtosis) that are catastrophically amplified by diluted softmax attention at long context, while a different local minimum with similar MSE but lower kurtosis handles long context gracefully.

4.4 Query Anisotropy and Theoretical Error Ceiling

We measure the covariance structure of query vectors in the rotated (post-FWHT) domain and find extreme anisotropy:

Proposition 1 (Q Anisotropy). *For Qwen3.5-27B, the rotated query covariance Σ_q has effective rank ≈ 5 out of 128 dimensions, with eigenvalue ratio $\lambda_{\max}/\lambda_{\min} \approx 937$. The dominant principal component captures $\sim 96\%$ of query variance.*

This has a striking implication for quantization: the attention dot product $\mathbf{q}^T\mathbf{k}$ is sensitive only to key errors along Σ_q's principal directions. The expected attention error is:

$$\mathbb{E}\left[\mathrm{Var}_j(\mathbf{q}^T\mathbf{e}_j)\right] = \mathrm{tr}(\Sigma_q \cdot \Sigma_e) \tag{15}$$

where Σ_e is the quantization error covariance. If errors could be concentrated in the 123 directions orthogonal to Σ_q's principal subspace, attention error would decrease by $\sim 25\times$ at the same MSE.

However, we find that TCQ quantization errors are *isotropic* in Σ_q's eigenvector basis: the ratio of error variance along the top-5 query eigenvectors to average error variance is 1.00 ± 0.01 for all tested codebooks. The FWHT rotation that enables analytical codebook design also destroys any structure that could be exploited for query-aware quantization.

Per-layer variation. The anisotropy varies dramatically across layers: layer 19 has effective rank 1.9 (nearly one-dimensional queries), while layer 20 has effective rank 114 (nearly isotropic). This suggests that per-layer codebook optimization — weighting errors by layer-specific Σ_q — could yield significant quality improvements, but is architecturally complex.

This finding connects to the Ordentlich-Polyanskiy result [6]: the optimal quantizer for the matrix multiplication Q^TK has a fundamentally different structure than the MSE-optimal quantizer, with density proportional to $(f(x) \cdot \mathbb{E}[Y^2|X = x])^{1/3}$ rather than $f(x)^{1/3}$. The $25\times$ theoretical ceiling represents the gap between our current isotropic TCQ and a hypothetical query-weighted quantizer.

5 Experiments

5.1 Setup

Model. We evaluate on Qwen3.5-27B (Q6_K quantized weights), a 27-billion parameter dense transformer with 40 layers, 24 query heads, 4 key-value heads, and head dimension $d = 256$. The large head dimension is favorable for rotation-based quantization: the post-FWHT distribution converges more tightly to Gaussian at $d = 256$ than at $d = 128$, and each 256-element vector provides a longer trellis path for TCQ to exploit.

Hardware. Primary measurements use an NVIDIA RTX 3090 (24 GB VRAM, Ampere SM86) for context lengths 2K–32K. Deep-context measurements (48K–128K) use an NVIDIA A100 (80 GB VRAM, Ampere SM80), as the KV cache exceeds 24 GB at these lengths. All measurements are single-GPU with full model offload. Cross-GPU results are reported separately, as we observe that quantization rankings are not stable across GPU architectures.

Dataset. WikiText-2 test set (raw tokens, 1.29 M bytes). We use maximum chunks at each context length (8 chunks at ≤16K, 4 chunks at 24K–32K, 2 chunks at 48K, 1 chunk at 64K–128K) to minimize variance from chunk sampling.

Metrics. Our primary metric is KL divergence between the output logits of the quantized KV cache and an FP16 (lossless) KV cache baseline, measured using `llama-perplexity -kl-divergence`. KLD directly measures how much the quantization changes the model's output distribution, independent of the downstream task. We also report perplexity (PPL) on WikiText-2 as a secondary metric, noting that PPL and KLD can diverge — a quantized model can achieve lower PPL than FP16 if the quantization acts as a regularizer (Section 4.2).

Base logits. For each context length, we generate reference logits using the same model with FP16 KV cache. Base logits are generated and measured on the same GPU to avoid cross-platform artifacts.

Table 3: KL divergence (lower = better) for TCQ vs non-TCQ at all context lengths. Compiled-in codebook, $\alpha_V = 1.04$. RTX 3090 (2K–32K), A100 (48K–128K).

Context	turbo3	turbo3_tcq	Δ	turbo2	turbo2_tcq	Δ
2K	0.0748	**0.0552**	−26.1%	0.1363	**0.1119**	−17.9%
4K	0.0635	**0.0571**	−10.2%	0.1401	**0.1063**	−24.1%
8K	0.0904	**0.0746**	−17.4%	0.1797	**0.1362**	−24.2%
16K	0.0920	**0.0702**	−23.7%	0.1862	**0.1300**	−30.2%
24K	0.0819	**0.0686**	−16.3%	0.1672	**0.1217**	−27.2%
32K	0.0576	**0.0447**	−22.3%	0.1383	**0.0914**	−33.9%
48K	0.0669	**0.0483**	−27.8%	0.1422	**0.0953**	−33.0%
64K	0.0699	**0.0553**	−20.9%	0.1532	**0.0999**	−34.8%
128K	0.0408	**0.0306**	−25.0%	0.1091	**0.0607**	−44.4%

Baselines. We compare six KV cache configurations:

- **f16**: Unquantized FP16 KV cache (reference).
- **q8_0**: 8-bit scalar quantization (llama.cpp built-in).
- **turbo3**: 3-bit TurboQuant vector quantization without TCQ.
- **turbo3_tcq**: 3-bit TurboQuant with trellis-coded quantization.
- **turbo2**: 2-bit TurboQuant without TCQ.
- **turbo2_tcq**: 2-bit TurboQuant with TCQ.

This isolates the contribution of TCQ: turbo3 vs turbo3_tcq and turbo2 vs turbo2_tcq differ *only* in whether the trellis code is applied.

Codebook configurations. TCQ uses a 512-state bitshift trellis with codebooks trained via the Generalized Lloyd Algorithm (Section 4.1). We evaluate both the analytically-derived compiled-in codebook and product-aware trained codebooks, reporting whichever achieves lower KLD at each context length. Non-TCQ types use the same FWHT rotation and norm extraction pipeline but with scalar Lloyd-Max quantization.

Alpha scaling. The norm reconstruction scale α is optimized per context length via grid search over $[0.96, 1.12]$, separately for keys and values (Section 3.2).

5.2 KLD Results

Table 3 shows our primary result: TCQ consistently reduces KL divergence by 10–44% relative to scalar quantization at the same bit rate.

Several patterns emerge from Table 3:

TCQ advantage is large and consistent. At 3-bit, TCQ reduces KLD by 10–28% across all context lengths. At 2-bit, the improvement is even larger: 18–44%, reflecting the greater coding gain available at lower rates where scalar quantization has fewer levels.

2-bit improvement grows with context. The 2-bit TCQ advantage generally increases from 17.9% at 2K to 44.4% at 128K tokens (with minor dips at 24K and 48K). This context-scaling effect is explained by CLT averaging (Section 4.3): at long contexts, the trellis code's structured error pattern averages more favorably than scalar quantization's independent errors.

Non-monotonic context scaling. KLD does not increase monotonically with context length. Instead, there is a spike at 8K followed by a decline to the lowest values at 32K and 128K. The 32K turbo3_tcq KLD (0.0447) is *lower* than the 2K value (0.0552), confirming that CLT averaging in the attention mechanism can more than compensate for the increased sequence length. This pattern is consistent across all quantization types including q8_0, suggesting it is a property of the evaluation data rather than the quantization.

Table 4: Perplexity (lower = better) on WikiText-2. RTX 3090 (2K, 32K), A100 (48K–128K). turbo3_tcq achieves *lower* PPL than FP16 at every context length.

Type	2K	32K	48K	128K
f16	5.805	6.950	6.286	5.540
q8_0	5.839	6.951	6.271	5.526
turbo3	5.850	7.088	6.346	5.573
turbo3_tcq	**5.777**	**6.862**	**6.178**	**5.461**
turbo2	6.079	7.492	6.835	6.042
turbo2_tcq	**6.005**	**7.099**	**6.470**	**5.609**

Table 5: 3-bit KLD across context lengths for top codebooks ($\alpha_V = 1.04$). Bold = best at each context. The optimal codebook shifts with context length.

Codebook	2K	4K	8K	16K	24K	32K
compiled-in	.0552	.0571	**.0746**	.0702	.0686	.0447
pm/iter080	**.0513**	.0567	.0779	.0697	**.0630**	.0442
pm/iter090	.0595	.0519	.0788	.0710	.0638	**.0419**
pm/iter060	.0593	**.0517**	.0793	.0694	.0663	.0461

5.3 Perplexity Results

Table 4 reports perplexity on WikiText-2 at short and deep context lengths.

turbo3_tcq beats lossless FP16. At every context length measured, 3-bit TCQ achieves lower perplexity than uncompressed FP16 KV cache. This counterintuitive result arises from the norm scaling factor $\alpha_V = 1.04$: the slight upward scaling of value norms acts as an implicit regularizer, sharpening the attention distribution and suppressing the contribution of noisy low-weight tokens. The improvement grows with context: −0.47% at 2K, −1.26% at 32K, −1.71% at 48K, −1.42% at 128K.

2-bit TCQ closes the gap. Without TCQ, 2-bit quantization incurs a 4.7–9.1% PPL penalty relative to FP16. With TCQ, this shrinks to 1.2–3.5%. At 128K, turbo2_tcq (5.609) is within 1.24% of FP16 (5.540) — a practical operating point for memory-constrained deployments.

5.4 Codebook Selection Across Contexts

We trained codebooks using four variants of the Generalized Lloyd Algorithm: *vanilla* (standard GLA), *mono* (monotonicity-constrained), *product* (product-aware distortion weighting), and *product_mono* (both constraints). Each variant was trained for 100 iterations, producing 400 codebooks per bit width. Table 5 shows the 3-bit context scaling results for the top codebooks.

No single codebook wins everywhere. The compiled-in (analytical) codebook wins at 8K; product_mono/iter080 wins at 2K and 24K; product_mono/iter090 wins at 32K; product_mono/iter060 wins at 4K. However, the differences between codebooks are small (2–7%) relative to the TCQ vs non-TCQ gap (17–27%). For deployment, product_mono/iter080 achieves the best average KLD (0.0605 vs 0.0617 for compiled-in, −2.0%).

At 2-bit (Table 6), codebook training has a larger effect, with the best trained codebook improving average KLD by 3.5%.

Product-aware training dominates. The product_mono training method — which jointly optimizes for the product distortion metric used in attention while enforcing monotonic coset structure — produces the best codebooks at both bit widths. This is consistent with the Ordentlich-Polyanskiy finding that MSE

Table 6: 2-bit KLD across context lengths for top codebooks ($\alpha_V = 1.04$). Trained codebooks provide larger improvements than at 3-bit.

Codebook	2K	4K	8K	16K	24K	32K
compiled-in	.1119	.1063	.1362	**.1300**	.1217	.0914
pm/iter090	**.0941**	.1016	.1320	.1352	**.1200**	.0901
vanilla/iter060	.0951	**.0957**	**.1283**	.1378	.1279	.0932
pm/iter080	.0964	.1130	.1307	.1353	.1278	**.0894**

Table 7: Optimal norm scaling α_V^* by context length. 3-bit alpha *decreases* with context; 2-bit alpha *increases* — opposite trends driven by different error regimes. Δ = improvement over default $\alpha = 1.04$.

	3-bit (pm/iter080)			2-bit (pm/iter090)		
Context	α^*	KLD	Δ	α^*	KLD	Δ
2K	1.04	0.0513	—	1.04	0.0941	—
8K	1.02	0.0743	−4.6%	1.08	0.1275	−3.4%
16K	1.02	0.0651	−6.6%	1.10	0.1255	−7.1%
32K	1.00	0.0402	−9.1%	1.10	0.0840	−6.7%
48K†	1.02	0.0471	−3.3%	1.08†	0.0877	−3.5%
64K†	1.00	0.0496	−6.9%	1.08†	0.0926	−7.3%

†A100 (SM80); all others RTX 3090 (SM86). 2-bit 48K–64K uses vanilla/iter060.

is suboptimal for matrix multiplication [6]: product-aware training steers codebooks toward attention-relevant distortion rather than pure MSE.

Convergence iteration differs by rate. The optimal GLA iteration is 80 for 3-bit and 90 for 2-bit. At 2-bit, more training iterations are needed because the smaller codebook (4 levels per coset vs 8) has less redundancy from the coset structure alone. This is consistent with the breakdown of UTCQ universality at low rates (Section 4.3).

5.5 Alpha Optimization

The norm scaling factor α_V (Section 3.2) controls the trade-off between quantization bias and variance. We sweep $\alpha_V \in \{0.98, 1.00, 1.02, 1.04, 1.06, 1.08, 1.10, 1.12\}$ at each context length for the selected codebooks. Table 7 shows the optimal alpha and the KLD improvement over the default $\alpha = 1.04$.

Opposite alpha trends at 3-bit and 2-bit. At 3-bit, the optimal alpha decreases monotonically from 1.04 at 2K to 1.00 at 32K–64K. At 2-bit, the opposite occurs: optimal alpha *increases* from 1.04 at 2K to 1.08–1.10 at longer contexts. This asymmetry has a natural interpretation. At 3-bit (8 centroids per coset), the trellis code already provides good coverage of the signal space, and the dominant error is bias from norm over-correction. Reducing α toward 1.0 removes this bias. At 2-bit (4 centroids per coset), the dominant error is variance from coarse quantization. Increasing α inflates the reconstructed norms, which amplifies the signal relative to the quantization noise — effectively performing a form of noise-aware Wiener filtering.

Alpha tuning is worth 3–9% KLD. The improvement from tuning alpha grows with context length, from negligible at 2K to 6–9% at 32K. We implement context-adaptive alpha using the log-linear and saturation formulas from Section 3.2, applied at decode time so that all tokens see the same alpha. This captures the full improvement from Table 7 automatically, with no manual tuning required.

Table 8: Decode throughput (tok/s, higher = better) on RTX 3090 at increasing KV cache depth. `tg128`, 3 repetitions.

Type	2K	8K	16K	32K
turbo3	28.24	26.14	24.32	21.72
turbo3_tcq	26.99	24.01	21.43	17.69
overhead	*−4.4%*	*−8.1%*	*−11.9%*	*−18.6%*
turbo2	28.30	26.49	24.88	22.31
turbo2_tcq	27.35	24.57	21.85	17.97
overhead	*−3.4%*	*−7.2%*	*−12.2%*	*−19.4%*

5.6 Deep Context (48K–128K)

Context lengths beyond 32K exceed the KV cache capacity of FP16 on a 24 GB GPU. Tables 3 and 4 include measurements at 48K, 64K, and 128K on an A100 80 GB GPU with separate same-GPU base logits (these numbers are not directly comparable to RTX 3090 measurements due to different float rounding). Deep context measurements use fewer chunks (2 at 48K, 1 at 64K–128K vs 8 at ≤16K) and consequently have higher variance.

Two findings extend from the shorter-context results:

TCQ advantage grows at deep context. The 2-bit TCQ improvement increases from 17.9% at 2K to 44.4% at 128K (Table 3). At 128K, turbo2_tcq achieves KLD 0.0607 — within 2× of turbo3 without TCQ (0.0408). The trellis code's structured errors average more favorably under the CLT effect in attention (Section 4.3), and this benefit compounds at longer sequences.

128K context on 24 GB. With turbo3 at 3.5 bpv, the KV cache for 128K tokens of Qwen3.5-27B occupies ∼4.7 GB, leaving ample headroom on a 24 GB GPU alongside the 20 GB model. FP16 KV cache would require ∼21 GB at 128K, exceeding the GPU budget. turbo2_tcq at 2.25 bpv reduces this further to ∼3.0 GB. In our competitive evaluation (Section 6), all four third-party TurboQuant forks failed at 32K on 24 GB — only our implementation succeeded.

5.7 Decode Speed

TCQ is not free. The sliding-window trellis decode (Section 3.3.4) replaces a direct 3-bit table lookup with a 9-bit state-indexed extraction per group of three values. Table 8 reports decode throughput at four context depths.

TCQ overhead grows with context depth. At 2K, the sliding-window decode adds only 3–4% overhead. At 32K, this grows to 18–19%. The reason is architectural: as the KV cache grows, the attention inner loop — where the trellis decode operates — occupies a larger fraction of total compute. At 2K, attention is a minor cost and the FFN layers dominate; at 32K, the KV cache read becomes the bottleneck and the trellis decode overhead is fully exposed.

Speed vs quality trade-off. At 8K (a typical deployment depth), TCQ costs ∼8% throughput but delivers 17–24% KLD improvement. At 32K, the cost grows to ∼19% for 22–34% KLD improvement. In competitive testing against four third-party TurboQuant implementations, our non-TCQ turbo3 is within 5% of the fastest fork (Madreag); the TCQ overhead is a cost no competitor pays because none of them implement trellis coding.

Memory efficiency. turbo3_tcq at 3.25 bpv uses 7% less memory than turbo3 at 3.5 bpv, and 59% less than q8_0 at 8 bpv. turbo2_tcq at 2.25 bpv provides 72% memory savings over q8_0. For a 27B model at 32K context, this is the difference between fitting on a 24 GB consumer GPU and requiring 48 GB+ server hardware.

6 Discussion and Conclusion

We have presented the first application of trellis-coded quantization to the KV cache of large language models. Our approach augments the existing TurboQuant vector quantization pipeline — FWHT rotation, Lloyd-Max scalar quantization, norm extraction — with a 512-state bitshift trellis that encodes each quantized value as a state transition rather than an independent index. The key results:

1. **17–44% KLD reduction** over scalar quantization at the same bit rate, across all context lengths from 2K to 128K (Table 3).
2. **turbo3_tcq beats FP16 in perplexity** at every context length measured (Table 4), achieving 3.5 bpv with better quality than lossless 16-bit storage.
3. **Context-adaptive alpha scaling** implemented as decode-time formulas with opposite trends at 3-bit (log-linear decrease) and 2-bit (saturation increase), automatically capturing the 3–9% KLD improvement from Table 7.
4. **O(1) decode** via sliding-window trellis extraction, with 4–19% throughput overhead depending on context depth (Table 8).
5. **128K context on 24 GB** consumer GPUs — a capability no 8-bit scheme can match on this model.

Why does quantization beat lossless? The turbo3_tcq PPL result (5.777 vs FP16's 5.805 at 2K) is surprising but has a clean explanation. The norm scaling factor α_V (1.04 at 2K context) slightly inflates value norms, which sharpens the attention distribution by amplifying the contribution of high-weight tokens relative to low-weight noise tokens. This is equivalent to a temperature reduction in the softmax. The trellis code further improves this by making quantization errors correlated across adjacent values, which creates a smoother error surface that the attention softmax can more easily absorb. The effect is a form of implicit regularization: the quantized model attends more precisely than the unquantized model, at the cost of a small distributional shift (KLD 0.055 at 2K).

Codebook universality. Our experiments confirm the theoretical prediction of Section 4.3: after FWHT rotation, the signal is sufficiently Gaussian that trained codebooks provide only 2–3.5% improvement over analytical codebooks at 3-bit. At 2-bit, where the Gaussianity assumption breaks down (fewer centroids = more sensitivity to tail behavior), training provides more benefit, and the product-aware distortion metric outperforms pure MSE — consistent with the Ordentlich-Polyanskiy finding that MSE is suboptimal for matrix multiplication [6].

Competitive landscape. We benchmarked four independent TurboQuant forks (TheTom, Madreag, AmesianX, Duster) on the same GPU with the same base logits. None implement TCQ. Our TCQ achieves 17–34% lower KLD than the best competitor at every context length. At 32K, all four competitors failed while our implementation succeeded. The speed gap is modest: the fastest competitor (Madreag) achieves ~5% higher throughput than our non-TCQ baseline, likely due to a larger block size (128 vs 32 elements). Our TCQ overhead (8% at 8K) is well within this margin.

Limitations. The TCQ decode overhead grows to 19% at 32K context, which may matter for latency-sensitive applications at long context. Prefill is also slower: the Viterbi encode path (512 states $\times$ 128 forward steps) adds ~21% overhead to prefill throughput. Our current implementation supports only GQA architectures with head_dim = 128 or 256; models with sliding-window attention (e.g., Gemma-3) require modifications to the V un-rotation path.

Future work. Three directions are immediately actionable. First, *asymmetric K/V quantization*: using 3-bit TCQ for keys (which the softmax exponentially amplifies) and 2-bit TCQ for values (which enter linearly) would achieve an effective 2.75 bpv. Preliminary PPL results are mixed — the optimal K/V rate allocation likely depends on context length and warrants systematic investigation. Second, *dynamic alpha refinement*: our context-adaptive alpha formulas (Section 3.2) capture the main trends, but per-layer or per-head alpha schedules may yield further gains. Second, *per-layer rate allocation*: early layers may tolerate

coarser quantization than later layers, enabling mixed 2-bit/3-bit schedules that optimize the overall rate-distortion trade-off.

Disclosure

This paper was written collaboratively by a human researcher and Claude, an AI assistant by Anthropic. Claude contributed to literature review, theoretical framing, drafting, and code review throughout the project. The research was vibed, but the benchmarks weren't — all experimental results were produced by real CUDA kernels running on real GPUs with no AI in the measurement loop.

References

[1] TurboQuant: Online Vector Quantization with Near-Optimal Distortion Rate. Google Research & NYU. ICLR 2026. arXiv:2504.19874.

[2] M. W. Marcellin and T. R. Fischer. Trellis coded quantization of memoryless and Gauss-Markov sources. *IEEE Trans. Comm.*, 38(1):82–93, 1990.

[3] T. R. Fischer and M. Wang. Entropy-constrained trellis coded quantization. *IEEE Trans. Inform. Theory*, 38(2):415–426, 1992.

[4] J. H. Kasner and M. W. Marcellin. Universal trellis coded quantization. *IEEE Trans. Image Processing*, 8(12):1677–1687, 1999.

[5] R. Zamir and M. Feder. On lattice quantization noise. *IEEE Trans. Inform. Theory*, 42(4):1152–1159, 1996.

[6] O. Ordentlich and Y. Polyanskiy. Optimal quantization for matrix multiplication. arXiv:2410.13780, 2024.

[7] AsymKV: Asymmetric quantization of key and value for KV cache compression. arXiv:2410.13212, 2024.

[8] M. J. Sabin and R. M. Gray. Global convergence and empirical consistency of the generalized Lloyd algorithm. *IEEE Trans. Inform. Theory*, 32(2):148–155, 1986.

[9] V. Kostina and S. Verdú. Fixed-length lossy compression in the finite blocklength regime. *IEEE Trans. Inform. Theory*, 58(6):3309–3338, 2012.

[10] A. Tseng et al. QTIP: Quantization with Trellises and Incoherence Processing. NeurIPS 2024. arXiv:2406.11235.

[11] J. Chee et al. QuIP#: Even Better LLM Quantization with Hadamard Incoherence and Lattice Codebooks. arXiv:2402.04396, 2024.

[12] A. Gersho. Asymptotically optimal block quantization. *IEEE Trans. Inform. Theory*, 25(4):373–380, 1979.

[13] R. M. Gray and D. L. Neuhoff. Quantization. *IEEE Trans. Inform. Theory*, 44(6):2325–2383, 1998.

[14] G. Gerganov et al. llama.cpp: LLM inference in C/C++. `https://github.com/ggerganov/llama.cpp`, 2023–2026.

[15] Z. Liu et al. KIVI: A tuning-free asymmetric 2bit quantization for KV cache. arXiv:2402.02750, 2024.

[16] C. Hooper et al. KVQuant: Towards 10 million context length LLM inference with KV cache quantization. arXiv:2401.18079, 2024.

[17] VecInfer: Outlier-suppressed vector quantization for memory-efficient LLM inference. arXiv:2510.06175, 2025.

[18] NSNQuant: Calibration-free VQ for KV cache via triple normalization. arXiv:2505.18231, 2025.

[19] KVTC: KV cache transform coding. NVIDIA Research. ICLR 2026. arXiv:2511.01815.

[20] More keys, less values: Asymmetric KV cache bit allocation. arXiv:2502.15075, 2025.

[21] RotateKV: 2-bit KV cache quantization via outlier-aware adaptive rotations. arXiv:2501.16383, 2025.

[22] MixQuant: Pushing the limits of block rotations in post-training quantization. arXiv:2601.22347, 2026.

[23] N. Ailon and B. Chazelle. The fast Johnson-Lindenstrauss transform and approximate nearest neighbors. *SIAM J. Comput.*, 39(1):302–322, 2009.

[24] T. Lookabaugh and R. M. Gray. High-resolution quantization theory and the vector quantizer advantage. *IEEE Trans. Inform. Theory*, 35(5):1020–1033, 1989.